

# Scheduling of a Flexible Flow Line with Limited Buffers and Bypass

Chapter 16.3 — Pinedo, *Scheduling: Theory, Algorithms, and Systems*

Berk Bozdağ (03825499)  
Nguyen Tam An Vo (03812033)  
Leon Ihrig (03827171)

July 15, 2026

## Abstract

This report analyzes the scheduling problem described in Section 16.3 of Pinedo’s *Scheduling: Theory, Algorithms, and Systems* [3]: the cyclic scheduling of a flexible flow line with limited buffers and job bypass, originally addressed at IBM via the Flexible Flow Line Loading (FFLL) algorithm [4]. We formalize the problem and its Graham three-field classification, derive and critically assess a mixed-integer programming (MIP) formulation with a linear time-indexed buffer constraint and an explicit machine-eligibility constraint, describe the three-phase FFLL heuristic and apply it to three minimal worked examples chosen to isolate specific behaviours (a case where FFLL is optimal, a minimal counter-example where it is not, and a case demonstrating machine eligibility restrictions), and benchmark FFLL against Gurobi’s exact solution of the MIP on a suite of small instances solved to proven optimality. Across the ten test instances, FFLL matches the exact optimum on five and averages a 6.8% gap to optimum overall – results that quantify the practical price of FFLL’s greedy, phase-decoupled structure relative to global optimization, while also confirming that the MIP itself is only tractable at small problem sizes, motivating heuristic use in industrial-scale settings.

## Contents

|          |                                                          |          |
|----------|----------------------------------------------------------|----------|
| <b>1</b> | <b>Problem Statement and Graham Classification</b>       | <b>3</b> |
| 1.1      | Problem Description . . . . .                            | 3        |
| 1.2      | Objectives and Alternative Objective Functions . . . . . | 3        |
| 1.3      | Graham Notation . . . . .                                | 4        |
| <b>2</b> | <b>MIP Formulation and Critical Assessment</b>           | <b>4</b> |
| 2.1      | Indices, Parameters and Decision Variables . . . . .     | 5        |
| 2.2      | MIP Model . . . . .                                      | 6        |
| 2.3      | Critical Assessment of the MIP . . . . .                 | 7        |
| <b>3</b> | <b>The Flexible Flow Line Loading Algorithm</b>          | <b>7</b> |
| 3.1      | Algorithm Description . . . . .                          | 7        |
| 3.2      | Minimal Application Examples . . . . .                   | 9        |
| 3.3      | Critical Assessment of the FFLL Heuristic . . . . .      | 12       |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>4</b> | <b>Instance Comparison: FFLL vs. Exact MIP</b> | <b>13</b> |
| 4.1      | Purpose and Setup . . . . .                    | 13        |
| 4.2      | MIP Implementation Notes . . . . .             | 13        |
| 4.3      | Results . . . . .                              | 13        |
| 4.4      | Practical Implications . . . . .               | 15        |

# 1 Problem Statement and Graham Classification

## 1.1 Problem Description

The problem considers a **flexible flow line (FFL)** – a production environment consisting of multiple stages arranged in series, where each stage contains one or more machines operating in parallel. The specific context in Section 16.3 of Pinedo [3] is the assembly of printed circuit boards (PCBs), an environment for which the Flexible Flow Line Loading (FFLL) algorithm was originally designed at IBM [4].

The environment has the following defining characteristics:

**Jobs** Each job represents a batch of relatively small identical items (e.g., PCBs). A set of jobs that together represent one full day’s production mix is called a *Minimum Part Set* (MPS). The manufacturing process is repetitive, so the MPS cycles through the system continuously.

**Processing** Each job must be processed at every stage on exactly one machine. Not all machines at a stage are necessarily identical – some may handle only specific job types.

**Bypass** A job may not require processing at every stage. If a job skips a stage, the material handling system transports it past that stage. This is modeled by setting the processing time of that job at the bypassed stage to zero,  $p_{ij} = 0$ .

**Limited buffers** Between stages, buffer space is finite. If a buffer is full, the material handling system halts – or, if recirculation is possible, the job bypasses the stage and recirculates.

**Cyclic schedule** Because production is repetitive, the objective is a *cyclic schedule*: the best ordering and timing of jobs within one MPS cycle, repeated indefinitely.

## 1.2 Objectives and Alternative Objective Functions

The FFLL algorithm pursues two objectives simultaneously [3]:

- **Primary objective – maximize throughput.** In cyclic scheduling this is equivalent to minimizing the cycle time of one MPS. The cycle time cannot be smaller than the workload of the bottleneck machine, so the algorithm seeks to approach this lower bound.
- **Secondary objective – minimize work-in-process (WIP).** Since buffer space is limited, reducing WIP lowers the probability of buffer overflow and the resulting blocking.

Alternative objective functions that could be substituted for  $C_{\max}$  include:

- Minimizing weighted tardiness  $\sum w_j T_j$  – relevant when jobs carry due dates and differing priorities.
- Minimizing total/average flow time  $\sum C_j$  – reducing the average time a job spends in the system.
- Minimizing the number of tardy jobs  $\sum U_j$  – relevant in customer-facing settings.

### 1.3 Graham Notation

We use the standard three-field notation  $\alpha \mid \beta \mid \gamma$ , presented by Pinedo [3] as an adapted version of the classical framework, together with Pinedo’s own machine-environment symbol for this specific class of problems. Pinedo denotes the flexible flow shop by **FFc**, where  $c$  is the number of stages in series, each stage being a bank of *parallel machines* [3]. The classification is therefore:

$$FFc \mid M_j, b_i < \infty \mid C_{\max}$$

| Field                          | Symbol         | Meaning                                                                                                            |
|--------------------------------|----------------|--------------------------------------------------------------------------------------------------------------------|
| $\alpha$ (machine environment) | $FFc$          | Flexible flow shop: $c$ stages in series, each stage a bank of parallel machines [3]                               |
| $\beta$ (constraints)          | $M_j$          | Machine eligibility restrictions: job $j$ may only be processed on a subset $M_j$ of the machines at its stage [3] |
|                                | $b_i < \infty$ | Finite buffer capacity between stages [2]                                                                          |
| $\gamma$ (objective)           | $C_{\max}$     | Minimize makespan / cycle time of the MPS                                                                          |

Table 1: Graham three-field classification of the flexible flow line scheduling problem.

**On omitting an informal “bypass” field entry.** Bypass is not a standard Graham symbol in the first place, and the two genuine structural features an informal “*bypass*” term might seem to require are already covered by entries that *are* standard. A job that bypasses a stage is, in our data, simply a job with  $p_{ij} = 0$  for every machine  $i$  at that stage – a property of the input data  $p_{ij}$ , not an additional constraint requiring its own  $\beta$ -symbol (Pinedo’s own chapter never introduces one either). Once machine eligibility is made explicit via the standard symbol  $M_j$  [3], where in the most general case  $M_j$  may be an arbitrary subset of the stage’s machines, the routing flexibility that an informal “bypass” notation would gesture at is already subsumed:  $M_j$  together with the convention  $p_{ij} = 0$  for bypassed stages fully describes which machines (if any) a job visits at a given stage, without requiring a separate notation symbol. The classification above is therefore both more compact and more faithful to standard usage.

**On the buffer-capacity entry.** The entry  $b_i < \infty$  is kept exactly as a finite-buffer-capacity statement, the most direct reading of the problem in Section 16.3. We attribute it to McCormick et al. [2], whose paper establishes the key structural fact connecting limited-buffer and zero-buffer (blocking) flow shops: *any* flow shop with a limited buffer of capacity  $b$  between two stages can be represented exactly by inserting  $b$  zero-processing-time “buffer machines” between them, reducing the limited-buffer case to the zero-buffer (blocking) case studied extensively in the flow-shop literature. Citing this reduction anchors the finite-buffer entry to a specific, well-established result rather than treating  $b_i < \infty$  as a bare descriptive shorthand.

**Remark.**  $FFc$  generalizes both the classical flow shop ( $Fm$ , single machine per stage) and the parallel-machine environment ( $Pm$ , a single stage); it reduces to  $Fm$  when every stage has exactly one machine. Machine eligibility restrictions ( $M_j$ ) mean routing is no longer uniform across jobs and finite buffers ( $b_i < \infty$ ) introduce *blocking*. Even the two-machine flow shop with blocking is known to be NP-hard [1], and the combination of parallel machines, machine eligibility, and finite buffers studied here is at least as hard. This NP-hardness is the central motivation for the heuristic (rather than exact) solution approach of Section 3.

## 2 MIP Formulation and Critical Assessment

Pinedo’s chapter does not present an explicit mixed-integer program; we construct one here that mirrors the three decisions made sequentially by the FFL algorithm: machine assignment, job sequencing, and timing.

## 2.1 Indices, Parameters and Decision Variables

### Indices and parameters.

- $n$ : number of jobs in the MPS, indexed  $j = 1, \dots, n$ .
- $m$ : total number of machines, indexed  $i = 1, \dots, m$ .
- $S$ : number of stages, indexed  $s = 1, \dots, S$ .
- $M_s$ : set of machines belonging to stage  $s$ .
- $p_{ij}$ : processing time of job  $j$  on machine  $i$  (0 if job  $j$  bypasses the stage containing machine  $i$ ).
- $E_{ij} \in \{0, 1\}$ : *eligibility* indicator,  $E_{ij} = 1$  if machine  $i$  is capable of processing job  $j$  and 0 otherwise. This captures the fact, noted explicitly in the problem description, that not all machines within a stage are identical and a given job may be restricted to a specific subset of machines at that stage.
- $b_s$ : buffer capacity between stage  $s$  and stage  $s + 1$ .
- $H$ : a finite planning horizon long enough to contain any feasible schedule of one MPS, e.g.  $H = \sum_{j=1}^n \sum_{i=1}^m p_{ij}$  (the sum of all processing times). Time is discretized into unit periods  $t \in \mathcal{T} = \{0, 1, \dots, H\}$ .
- $L$ : a sufficiently large constant (big- $M$ ), e.g.  $L = H$ .

We also use the following derived set, defined purely in terms of the parameters above (not an independent input to the model):

$$M_s(j) := \{i \in M_s : E_{ij} = 1\}, \quad \text{the machines at stage } s \text{ eligible to process job } j.$$

By convention, if job  $j$  bypasses stage  $s$  entirely then  $E_{ij} = 1$  for all  $i \in M_s$  and  $p_{ij} = 0$  for all  $i \in M_s$  – bypass and eligibility are independent concepts and must not be conflated: bypass means the job skips the *stage*, whereas  $E_{ij} = 0$  means the job must avoid a *specific machine* while still being processed somewhere else at that same stage. The convention  $E_{ij} = 1$  for all  $i \in M_s$  during bypass allows the machine-assignment constraint (C1a) to still fire, selecting a zero-duration “ghost” visit that keeps the series-precedence chain (C3a) intact across the skipped stage.

| Variable   | Domain                | Meaning                                                                                       |
|------------|-----------------------|-----------------------------------------------------------------------------------------------|
| $x_{ij}$   | $\{0, 1\}$            | 1 if job $j$ is assigned to machine $i$                                                       |
| $y_{ijj'}$ | $\{0, 1\}$            | Defined for $j < j'$ : 1 if job $j$ precedes job $j'$ on machine $i$ , else 0                 |
| $C_{ij}$   | $\mathbb{R}_{\geq 0}$ | Completion time of job $j$ on machine $i$                                                     |
| $a_{jst}$  | $\{0, 1\}$            | 1 if job $j$ has departed its stage- $s$ machine by time $t$                                  |
| $b_{jst}$  | $\{0, 1\}$            | 1 if job $j$ has not yet started its stage- $(s+1)$ machine by time $t$                       |
| $z_{jst}$  | $\{0, 1\}$            | 1 if job $j$ is waiting in the buffer between stage $s$ and $s+1$ at time $t \in \mathcal{T}$ |
| $W_i$      | $\mathbb{R}_{\geq 0}$ | Total workload assigned to machine $i$ , defined by (C5a)                                     |
| $T$        | $\mathbb{R}_{\geq 0}$ | Cycle time (makespan of one MPS)                                                              |

### Decision variables.

## 2.2 MIP Model

$$\begin{aligned}
& \text{minimize } T & (1) \\
& \text{s.t. } \sum_{i \in M_s(j)} x_{ij} = 1 & \forall j, \forall s & (\text{C1a}) \\
& x_{ij} \leq E_{ij} & \forall i, \forall j & (\text{C1b}) \\
& C_{ij'} \geq C_{ij} + p_{ij'} - L(1 - y_{ijj'}) - L(2 - x_{ij} - x_{ij'}) & \forall i, \forall j < j' & (\text{C2a}) \\
& C_{ij} \geq C_{ij'} + p_{ij} - L y_{ijj'} - L(2 - x_{ij} - x_{ij'}) & \forall i, \forall j < j' & (\text{C2b}) \\
& C_{ij} \geq C_{i'j} + p_{ij} x_{ij} - L(2 - x_{ij} - x_{i'j}) & \forall j, s, i \in M_s, i' \in M_{s-1} & (\text{C3a}) \\
& C_{ij} \geq p_{ij} x_{ij} & \forall i, j & (\text{C3b}) \\
& t - C_{i'j} \leq L a_{jst} + L(1 - x_{i'j}) & \forall j, s, i' \in M_s, t \in \mathcal{T} & (\text{C4a}) \\
& C_{i'j} - t \leq L(1 - a_{jst}) + L(1 - x_{i'j}) & \forall j, s, i' \in M_s, t \in \mathcal{T} & (\text{C4b}) \\
& C_{ij} - (t + 1) \leq L b_{jst} + L(1 - x_{ij}) & \forall j, s, i \in M_{s+1}, t \in \mathcal{T} & (\text{C4c}) \\
& (t + 1) - C_{ij} \leq L(1 - b_{jst}) + L(1 - x_{ij}) & \forall j, s, i \in M_{s+1}, t \in \mathcal{T} & (\text{C4d}) \\
& z_{jst} \geq a_{jst} + b_{jst} - 1 & \forall j, s, t \in \mathcal{T} & (\text{C4e}) \\
& \sum_{j=1}^n z_{jst} \leq b_s & \forall s, \forall t \in \mathcal{T} & (\text{C4f}) \\
& W_i = \sum_{j=1}^n p_{ij} x_{ij} & \forall i & (\text{C5a}) \\
& T \geq W_i & \forall i & (\text{C5b}) \\
& x_{ij}, y_{ijj'}, a_{jst}, b_{jst}, z_{jst} \in \{0, 1\}, \quad C_{ij}, W_i, T \geq 0 & & (\text{C6})
\end{aligned}$$

(C1a) assigns every job to exactly one *eligible* machine per stage – restricting the summation to  $M_s(j)$  rather than the full stage  $M_s$  already excludes ineligible machines from the feasible region, and bypassed stages satisfy it trivially since  $p_{ij} = 0$  for all  $i \in M_s(j)$ . (C1b) forbids assigning job  $j$  to any machine  $i$  with  $E_{ij} = 0$ , regardless of processing time. (C2a–C2b) are the classical disjunctive constraints preventing two jobs from overlapping on the same machine; the additional  $-L(2 - x_{ij} - x_{ij'})$  term relaxes both constraints to slack whenever either job is not assigned to machine  $i$ , so they only fire on genuine same-machine job pairs. (C3a) enforces the series structure of the flow line: if job  $j$  is assigned to both an upstream machine  $i'$  at stage  $s - 1$  and a downstream machine  $i$  at stage  $s$ , its stage- $s$  completion cannot precede its stage- $(s - 1)$  completion by less than its processing time; again the big- $L$  term relaxes the constraint when either assignment is inactive. (C3b) sets a lower bound on the completion time of each assigned job: a job cannot complete before at least its own processing time has elapsed (critical for jobs that are the sole occupant of their machine, where the disjunctive constraints alone would leave  $C_{ij}$  under-determined).

(C4a–C4f) jointly enforce the buffer capacity. The binary  $z_{jst}$  should equal 1 when job  $j$  is physically occupying the buffer between stage  $s$  and  $s + 1$  at discrete time  $t$  – that is, when  $j$  has departed its stage- $s$  machine ( $C_{i'j} \leq t$ ) and has not yet started its stage- $(s+1)$  machine ( $t < C_{ij}$ , equivalently  $C_{ij} \geq t + 1$  for integer time). Since this is a conjunction of two threshold conditions, each condition is captured by its own auxiliary binary and its own pair of big- $L$  inequalities. (C4a–C4b) pin down  $a_{jst}$  to equal 1 if and only if  $C_{i'j} \leq t$  (departure): (C4a) forces  $a_{jst} = 1$  whenever the job has departed (since the nonnegative gap  $t - C_{i'j}$  must be covered by  $L \cdot a_{jst}$ ), and (C4b) symmetrically forces  $a_{jst} = 0$  whenever the job has not departed. The additional  $L(1 - x_{i'j})$  slack term deactivates both constraints unless the job is actually assigned to that upstream machine. (C4c–C4d) symmetrically pin down  $b_{jst}$  to equal 1 if and only if  $C_{ij} \geq t + 1$  (job has not yet arrived at the downstream machine), with the same  $L(1 - x_{ij})$  slack for the downstream assignment. In every big- $L$  pair the constant  $L$  multiplies the binary being fixed, which is what allows the constraint

to relax completely on the side that should not bind. (C4e) is the standard  $z_{jst} \geq a_{jst} + b_{jst} - 1$ , which forces  $z_{jst} = 1$  exactly when both conditions hold. Only the lower bound is imposed because  $z_{jst}$  appears only in (C4f) as a nonnegative summand and nothing rewards a solver for setting it to 1 outside the buffer window. (C4f) then caps, at every discrete time  $t \in \mathcal{T}$ , the number of jobs simultaneously occupying the buffer between stage  $s$  and  $s + 1$  at  $b_s$ . (C5a) explicitly defines the workload variable  $W_i$  as the sum of processing times assigned to machine  $i$  under the current allocation, and (C5b) is the valid inequality stating that the cycle time cannot fall below the workload of any single machine. The combined effect of (C5a)–(C5b) is a bottleneck lower bound that tightens the LP relaxation considerably.

## 2.3 Critical Assessment of the MIP

### Strengths.

- Provides an exact representation; guarantees global optimality given sufficient solver time.
- Flexible: alternative objectives (WIP, weighted tardiness) can be substituted directly into (1).
- Useful as an exact benchmark for evaluating heuristic solution quality on small instances.
- Machine eligibility (C1b) – the requirement that some jobs be restricted to specific machines within a stage – is captured at negligible cost, since it is a simple variable bound rather than a disjunctive or time-indexed constraint.

### Weaknesses.

- The buffer constraints (C4a)–(C4f), require three binary variables ( $a_{jst}$ ,  $b_{jst}$ , and  $z_{jst}$ ) for every (job, stage-boundary, time-period) triple to correctly model the condition while also causing the size to scale directly with the planning horizon  $H$ .
- The bypass feature, while trivial to encode via  $p_{ij} = 0$ , interacts subtly with the buffer constraints, since bypassing jobs still occupy the material handling system.
- The model does not scale: industrial PCB assembly lines may involve dozens of stages and hundreds of job types per MPS, far beyond the practical reach of exact MIP solvers.

**Overall verdict.** The MIP is theoretically sound and a useful small-instance benchmark, but it is not a practical solution method for industrial-scale instances of this problem – precisely the motivation behind IBM’s development of the FFLL heuristic [4].

## 3 The Flexible Flow Line Loading Algorithm

### 3.1 Algorithm Description

The Flexible Flow Line Loading (FFLL) algorithm [3, 4] solves the problem in three sequential phases, each addressing single core decisions: *who* processes *what*, in which *order*, and *when*.

#### Phase 1 — Machine Allocation (LPT Heuristic)

Each job is assigned to one machine per stage so that workloads across parallel machines at that stage are as balanced as possible, balance directly determines the bottleneck workload and hence the achievable cycle time. As noted in Section 1.1, not all machines within a stage are necessarily identical, and a job may be restricted to a subset  $M_s(j) \subseteq M_s$  of eligible machines at that stage (the same eligibility notion introduced for the MIP in Section 2.1). We therefore apply an *eligibility-restricted* Longest Processing Time (LPT) heuristic [3] independently at every stage with multiple machines:

**Algorithm 1:** The FFL algorithm (three-phase decomposition)

---

**input** : jobs  $j = 1, \dots, n$ ; stages  $s = 1, \dots, c$  with machine sets  $M_s$ ,  $m$  machines in total; eligible machines  $M_s(j) \subseteq M_s$ ; processing times  $p_{ij}$ ,  $i \in M_s(j)$  (bypass of stage  $s$  encoded as zero processing time at  $s$ ); buffer capacities  $b_s$  (respected in the Phase-3 simulation)

**output**: a machine allocation, a loading sequence  $\sigma$ , and start times

// Phase 1 - machine allocation (eligibility-restricted LPT)

```

1 foreach stage  $s$  with  $|M_s| > 1$  // single-machine stages are handled directly do
2   foreach job  $j$  requiring stage  $s$ , in decreasing order of processing time at  $s$  do
3     [ assign  $j$  to its currently least-loaded eligible machine,
        [ arg min $_{i \in M_s(j)}$ (current load of  $i$ )

// Phase 2 - sequencing (Dynamic Balancing)
4 compute the overloads  $o_{ij}$ 
5  $S_0 \leftarrow \emptyset$ 
6 for position  $j = 1, \dots, n$  do
7    $\sigma(j) \leftarrow \arg \min_{k \notin S_{j-1}} \sum_{i=1}^m \max(O_{ij}, 0)$  //  $O_{ij}$  evaluated with  $S_j = S_{j-1} \cup \{k\}$ ,
   cf. Eq. (2)
8    $S_j \leftarrow S_{j-1} \cup \{\sigma(j)\}$ 

// Phase 3 - timing of releases (bottleneck anchoring)
9  $i^* \leftarrow \arg \max_i W_i$  // bottleneck; cycle-time lower bound  $T^* = W_{i^*}$ 
10 release all jobs in order  $\sigma$  as fast as possible and simulate the schedule
11 fix the start/completion times on  $i^*$ ; no idle time is permitted there
12 upstream of  $i^*$ : delay processing as much as possible without altering job sequences (reduces WIP)
13 downstream of  $i^*$ : process jobs as early as possible, again without altering sequences

```

---

1. Sort all jobs in decreasing order of processing time at that stage.
2. Assign each job, one at a time, to the currently least-loaded *eligible* machine at that stage, i.e.  $\arg \min_{i \in M_s(j)}(\text{current load of } i)$ , rather than the least-loaded machine overall.
3. Repeat until all jobs are assigned.

When every job is eligible for every machine at a stage ( $M_s(j) = M_s \forall j$ ), this reduces exactly to introduced example in the Pinedo chapter. When eligibility is restrictive, e.g. only one machine at a stage can run a particular job, the heuristic could be altered to assign that job to its sole eligible machine, regardless of load; balance is then only achievable among whatever jobs retain a real choice.

**Phase 2 — Sequencing (Dynamic Balancing Heuristic)**

Let  $n$  be the number of jobs and  $m$  the number of machines. Define



$$\begin{aligned}
W_i &= \sum_{j=1}^n p_{ij} && \text{workload of machine } i, \\
W &= \sum_{i=1}^m W_i && \text{total system workload,} \\
p_k &= \sum_{i=1}^m p_{ik} && \text{system-wide workload of job } k, \\
o_{ij} &= p_{ij} - p_j \frac{W_i}{W} && \text{overload contribution of job } j \text{ to machine } i, \\
O_{ij} &= \sum_{k \in S_j} o_{ik} && \text{cumulative overload on machine } i \text{ after position } j,
\end{aligned}$$

where  $S_j$  is the set of jobs loaded up to and including position  $j$  in the sequence. The Dynamic Balancing heuristic greedily minimizes

$$\sum_{i=1}^m \sum_{j=1}^n \max(O_{ij}, 0) \quad (2)$$

by, at each position, selecting from the remaining unscheduled jobs the one that minimizes the resulting total positive cumulative overload.

### Phase 3 — Timing of Releases (Bottleneck Anchoring)

1. Identify the bottleneck machine  $i^* = \arg \max_i W_i$ ; the cycle time lower bound is  $T^* = W_{i^*}$ .
2. Release all jobs into the system as rapidly as possible and simulate the resulting schedule.
3. Fix the start/completion times on the bottleneck machine, no idle time is permitted there.
4. For machines *upstream* of the bottleneck, delay processing as much as possible without altering job sequences, reducing WIP.
5. For machines *downstream* of the bottleneck, process jobs as early as possible, again without altering sequences.

## 3.2 Minimal Application Examples

The following examples are constructed as small as possible while still isolating a specific behaviour of the algorithm. Example A is the smallest instance on which FFLL matches the MIP-optimal makespan; Example B is the smallest instance on which it does not (demonstrating a clean structural failure mode); and Example C demonstrates the machine-eligibility mechanism of Sections 2.1 and 3.1.

### Example A — FFLL Matches the Optimum (Minimal Good Case)

Three identical jobs, two stages, machines-per-stage (2, 1); no bypass, no eligibility restrictions. All three jobs have processing time 2 at Stage 1 and 1 at Stage 2.

**Phase 1 (LPT at Stage 1).** Sorted descending: all three jobs tied at  $p' = 2$ . Assignment fills the two Stage-1 machines round-robin: Job 1→M1 (load 2), Job 2→M2 (load 2), Job 3→M1 (load 4). Machine 3 (Stage 2) processes all three jobs,  $W_3 = 3$ . Machine workloads:  $W = (4, 2, 3)$ ; bottleneck is M1 with  $T_{LB}^* = 4$ .

**Phase 2 (Dynamic Balancing).** All jobs are identical: their per-machine overload contributions are equal, so every position is a tie and any permutation is chosen. The heuristic returns  $1 \rightarrow 2 \rightarrow 3$ .

|         | Job 1 | Job 2 | Job 3 |
|---------|-------|-------|-------|
| Stage 1 | 2     | 2     | 2     |
| Stage 2 | 1     | 1     | 1     |

Table 2: Stage processing times  $p'_{kj}$  for Example A.

**Phase 3 (Timing).** Stage 1 M1: Job 1 processes 0–2, Job 3 processes 2–4. M2: Job 2 processes 0–2. Stage 2 M3: Job 2 arrives at time 2, processes 2–3; Job 1 arrives at time 2, waits for M3, processes 3–4; Job 3 arrives at time 4, processes 4–5. The achieved makespan is 5, which matches the MIP-optimal makespan for this instance (verified by direct Gurobi solve).

*Why FFL is optimal here.* The instance has no structural asymmetry that a smarter sequencing rule could exploit: because all jobs have identical processing profiles, no permutation of them can change the schedule’s shape. FFL’s greedy Phase 2 therefore cannot be wrong. This is the qualitative envelope of “FFL works well”: whenever the differences between jobs are small enough that they do not create a meaningful sequencing decision, greedy is optimal.

### Example B — FFL Falls Short of the Optimum (Minimal Counter-Example)

Three jobs, two stages, one machine per stage; no bypass, no eligibility restrictions. Total processing time is 8 units. This is a minimal instance on which FFL’s Phase 2 makes an unambiguous (non-tied) choice that turns out to be strictly suboptimal.

|         | Job 1 | Job 2 | Job 3 |
|---------|-------|-------|-------|
| Stage 1 | 1     | 2     | 1     |
| Stage 2 | 1     | 1     | 2     |

Table 3: Stage processing times  $p'_{kj}$  for Example B.

Job 1 is small on both stages; Job 2 is heavy on Stage 1 and light on Stage 2; Job 3 is the reverse.

**Phase 1 (LPT).** Trivial with only one machine per stage; both jobs go to the sole machine at each stage. Machine workloads:  $W = (4, 4)$ ,  $\sum W = 8$ ; bottleneck lower bound  $T_{LB}^* = 4$ .

**Phase 2 (Dynamic Balancing).** With  $p_j = (2, 3, 3)$  and  $W = (4, 4)$ , the overload contributions  $o_{ij} = p_{ij} - p_j \cdot W_i / \sum W$  evaluate to

$$o = \begin{pmatrix} 0 & +1/2 & -1/2 \\ 0 & -1/2 & +1/2 \end{pmatrix}.$$

Position 1: Job 1 gives cumulative-overload score 0; Job 2 and Job 3 each give score 1/2. Job 1 wins unambiguously. Position 2: starting from cumulative overload (0, 0), Job 2 and Job 3 give identical scores 1/2; the tie resolves to Job 2 (first-tested wins). Sequence: **1** → **2** → **3**.

**Phase 3 (Timing).** Stage 1: Job 1 processes 0–1, Job 2 processes 1–3, Job 3 processes 3–4. Stage 2: Job 1 arrives at 1, processes 1–2; Job 2 arrives at 3, processes 3–4; Job 3 arrives at 4, processes 4–6. Makespan is **6**.

**Optimum.** The sequence **1** → **3** → **2** instead gives Stage 1: Job 1 processes 0–1, Job 3 processes 1–2, Job 2 processes 2–4; Stage 2: Job 1 processes 1–2, Job 3 processes 2–4, Job 2 processes 4–5. Makespan is **5**. Gurobi confirms  $T^* = 5$ . FFL’s gap is  $6/5 - 1 = 20\%$ .

*Why FFL falls short here.* FFL puts Job 2 into position 2 even though Job 3 would have been the better choice. The reason is that Dynamic Balancing’s overload metric  $o_{ij}$  evaluates each job’s contribution to *workload balance* in isolation and treats Jobs 2 and 3 as symmetric, they contribute equal-magnitude overloads on the two machines, so their per-position scores are identical. But Jobs 2 and 3 are *not* interchangeable in the schedule: Job 3, with its small Stage 1 time and large Stage 2 time, should go earlier to keep the Stage-2 machine continuously busy (the classical Johnson’s-rule intuition for two-machine flow shops). The overload metric encodes no such stage-interaction

awareness, and this minimal counter-example exposes that blind spot as directly as possible. Any instance whose optimal sequence depends on stage-interaction effects rather than pure workload balance will produce a similar gap.

### Example C — Two Stages, Six Jobs, with General Machine Eligibility Restrictions

Eligibility restrictions in practice are not limited to a simple "one restricted machine vs. one universal machine" pattern: at a stage with several machines, *each* job can have its own eligible subset, of any size, independent of the other jobs' subsets. This example demonstrates that general case directly: Stage 1 has *four* parallel machines, and the six jobs have eligible-machine counts ranging from 1 of 4 up to 4 of 4. Stage 2 has a single machine, so eligibility there is moot.

|         | Job 1 | Job 2 | Job 3 | Job 4 | Job 5 | Job 6 |
|---------|-------|-------|-------|-------|-------|-------|
| Stage 1 | 6     | 5     | 7     | 4     | 8     | 3     |
| Stage 2 | 2     | 3     | 2     | 4     | 3     | 2     |

Table 4: Stage processing times  $p'_{kj}$  for Example C.

| Job | Eligible machines at Stage 1        |
|-----|-------------------------------------|
| 1   | {1, 2, 3} (3 of 4)                  |
| 2   | {2, 4} (2 of 4)                     |
| 3   | {1, 3, 4} (3 of 4)                  |
| 4   | {3} (1 of 4)                        |
| 5   | {1, 2, 3, 4} (4 of 4, unrestricted) |
| 6   | {1, 4} (2 of 4)                     |

Table 5: Eligibility sets  $M_1(j)$  for Example C, illustrating that restrictions can affect any subset of machines, of any size, not merely a single "restricted machine vs. universal machine" pattern.

**Phase 1 (eligibility-restricted LPT at Stage 1).** Sorting descending by Stage-1 time: Job 5 (8), Job 3 (7), Job 1 (6), Job 2 (5), Job 4 (4), Job 6 (3). We track the load on each of the four machines as the algorithm proceeds, restricting the "least-loaded" choice at each step to that job's eligible set:

1. **Job 5** (eligible: {1, 2, 3, 4}, all tied at load 0)  $\rightarrow$  Machine 1 (load 0  $\rightarrow$  8).
2. **Job 3** (eligible: {1, 3, 4}; loads are 8,  $-$ , 0, 0 among eligible machines)  $\rightarrow$  Machine 3 (load 0  $\rightarrow$  7).
3. **Job 1** (eligible: {1, 2, 3}; loads are 8, 0, 7)  $\rightarrow$  Machine 2, the least loaded eligible machine (load 0  $\rightarrow$  6).
4. **Job 2** (eligible: {2, 4}; loads are 6, 0)  $\rightarrow$  Machine 4 (load 0  $\rightarrow$  5).
5. **Job 4** (eligible: {3} *only*)  $\rightarrow$  Machine 3 by necessity (load 7  $\rightarrow$  11), even though Machine 3 is already the most heavily loaded eligible machine for Job 1 a moment earlier and is, after this step, the most heavily loaded machine at the stage overall. This is the general case of a job with only one eligible machine out of several: it must be placed there regardless of current load.
6. **Job 6** (eligible: {1, 4}; loads are 8, 5)  $\rightarrow$  Machine 4 (load 5  $\rightarrow$  8).

The resulting Stage-1 workloads are  $W_{1:4} = (8, 6, 11, 8)$ . Job 4's forced placement on Machine 3, the single machine it is eligible for, is what pushes Machine 3 to 11, well above the other three

machines; no amount of cleverness in ordering the other jobs could have prevented this, since Job 4 has no alternative. Including Stage 2 (a single machine processing all six jobs, total load 16), the full machine workload vector is  $W = (8, 6, 11, 8, 16)$ ,  $\sum W = 49$ . **Bottleneck: Machine 5 (Stage 2)**,  $T^* = 16$ .

**Contrast with the unrestricted case.** If eligibility were ignored entirely (every job eligible for every Stage-1 machine), LPT would instead produce  $W_{1:4} = (8, 7, 9, 9)$ , a range of just 2 (from 7 to 9) versus the range of 5 (from 6 to 11) obtained under the eligibility restriction above. The overall bottleneck is unaffected here (Machine 5 at Stage 2 dominates either way, since its load of 16 exceeds any Stage-1 machine's load in both cases), but Stage 1's *internal* balance is visibly worse under the restriction – exactly the effect described in Section 3.1, now shown with eligible-set sizes other than the trivial 1-of-2 case.

**Phase 2 (Dynamic Balancing).** Evaluating  $o_{ij}$  and applying the greedy rule position-by-position yields the sequence  $6 \rightarrow 4 \rightarrow 1 \rightarrow 5 \rightarrow 3 \rightarrow 2$ .

**Phase 3 (Timing).** The bottleneck Machine 5 processes jobs back-to-back in this order; the achieved makespan is 19.00, giving a gap of 18.8% to the lower bound  $T^* = 16$ .

### 3.3 Critical Assessment of the FFLL Heuristic

#### Strengths.

- Computationally efficient: Phase 1 runs in  $O(n \log n)$  per stage, Phase 2 in  $O(n^2 m)$ , and Phase 3 in linear time, tractable for hundreds of job types in industrial MPS instances.
- The three-phase decomposition mirrors the natural decision hierarchy: allocation must precede sequencing (workloads must be known first), and sequencing must precede timing.
- The Dynamic Balancing objective (2) has a clear theoretical motivation: minimizing burst loading directly reduces buffer occupancy and blocking probability.
- Bypass is handled naturally, since bypassed stages simply contribute zero to all  $o_{ij}$  terms.
- Machine eligibility restrictions are handled by a straightforward LPT modification that restricts each job's candidate machine to its eligible subset  $M_s(j)$ , at no extra computational cost.
- Extensive practical experiments reported by Pinedo confirm FFLL's value as a scheduling tool in real assembly environments [3].

#### Limitations.

- The three phases are solved *independently and sequentially* with no feedback loop, a poor Phase 1 allocation cannot be corrected by Phases 2 or 3.
- LPT does not guarantee optimal balance; which is noted explicitly in Stage 3 of Example 16.3.1.
- Severe eligibility restrictions can defeat balancing entirely: if only one machine at a stage is eligible for a given job, that job must go there regardless of current load, and the achievable balance is limited to whatever flexibility remains among jobs with multiple machine options.
- The Dynamic Balancing heuristic is *greedy with no lookahead*: a locally optimal choice at position  $j$  may prevent a substantially better global sequence.
- WIP minimization is addressed only indirectly, through Phase 3's timing adjustments, with no explicit guarantee against blocking under tight buffers.
- The algorithm assumes a fixed, fully known MPS and offers no mechanism for incremental re-optimization under dynamic arrivals or uncertain processing times.

- As a greedy, non-backtracking heuristic, FFLL cannot revisit an early sequencing decision once later choices reveal it was poor. As the experimental comparison against exact optima in Section 4 will show, this greediness can leave a substantial optimality gap on instances whose structure rewards backtracking.

## 4 Instance Comparison: FFLL vs. Exact MIP

### 4.1 Purpose and Setup

To quantify how much makespan is left on the table by FFLL’s greedy, non-backtracking construction, we compare its output against the exact optimum produced by directly solving the MIP of Section 2 with a commercial branch-and-bound solver (Gurobi 13.0.2). Because the MIP scales poorly with instance size,  $O(mn^2)$  disjunctive constraints alone reach several hundred binaries for a modest instance, we deliberately restrict the comparison to *small* instances (3 to 5 jobs, 2 to 3 stages) so that every instance in the suite is solved to *provable* optimality within seconds. This trade lets us give up scale in exchange for a rigorous, guaranteed-tight benchmark against which FFLL’s gap can be measured without ambiguity.

Ten instances were generated with varied characteristics (jobs per MPS, stages, machines per stage, processing time ranges, and bypass probability). All processing times are drawn as integers from a bounded uniform distribution, and machines-per-stage are drawn from  $\{1, 2\}$ . Two instances (S9, S10) include bypass. The complete generator and instance parameters are in the accompanying source file `ffll_vs_mip.py`.

### 4.2 MIP Implementation Notes

The Gurobi implementation follows the formulation of Section 2 exactly, with two implementation notes worth stating for reproducibility.

*First*, the big- $L$  constant is set to  $L = H + 100$  where  $H = n \sum_{ij} p'_{ij}$ , an instance-specific but generous horizon; smaller  $L$  would tighten the LP relaxation and speed up branch-and-bound, but  $H + 100$  suffices at this scale.

*Second*, the sequencing binary  $y_{ijj'}$  is defined only for the unordered pair  $j < j'$  rather than every ordered pair. This halves both the number of sequencing binaries and the number of disjunctive constraints without changing the model’s correctness. The workload variables  $W_i$  from (C5a) are used for post-solve diagnostics such as per-machine workload reporting and Gantt-chart annotation.

### 4.3 Results

Table 6 reports the FFLL makespan, the MIP-optimal  $T^*$ , the resulting gap, and the Gurobi solve time for each of the ten instances.

**Findings.** *FFLL matches the optimum on 5 of 10 instances (S2–S5, S9) and comes within 10% on three more (S1, S6, S8). On the remaining two (S7 and S10) it leaves a substantial gap, up to 26.3% on S10. Across the full suite the average gap is 6.8%. Two observations explain where FFLL falls short.*

*First*, on instances where a bypass or stage-mismatch changes the effective bottleneck (S7, S8, S10), FFLL’s per-stage overload measure fails to look across stages: it can commit to a stage-1 order that appears well-balanced there but forces cascading idle time downstream.

*Second*, on instances with balanced stages or where the bottleneck aligns cleanly with a single machine’s total workload (S2–S5, S9), FFLL’s overload measure aligns closely with what the exact model optimizes and no gap opens up. FFLL is therefore *not uniformly suboptimal*: it is genuinely optimal, or nearly so, when the problem structure matches its assumptions. The gap emerges specifically when the interaction between stages combined with routing complexity (bypass, unbalanced machine counts) forces a decision that cannot be resolved by the per-phase greedy decomposition.

| #              | Config     | Jobs | Stg | Mach. per stage | FFLL  | MIP $T^*$    | Gap         | MIP time      |
|----------------|------------|------|-----|-----------------|-------|--------------|-------------|---------------|
| S1             | 3j-2s      | 3    | 2   | (1, 2)          | 12.00 | <b>11.00</b> | 9.1%        | 0.13 s        |
| S2             | 3j-2s      | 3    | 2   | (2, 1)          | 18.00 | 18.00        | 0.0%        | 0.18 s        |
| S3             | 4j-2s      | 4    | 2   | (2, 1)          | 22.00 | 22.00        | 0.0%        | 0.38 s        |
| S4             | 4j-2s      | 4    | 2   | (2, 2)          | 18.00 | 18.00        | 0.0%        | 1.23 s        |
| S5             | 4j-3s      | 4    | 3   | (2, 2, 1)       | 17.00 | 17.00        | 0.0%        | 0.59 s        |
| S6             | 5j-2s      | 5    | 2   | (1, 2)          | 25.00 | <b>24.00</b> | 4.2%        | 0.43 s        |
| S7             | 5j-2s      | 5    | 2   | (2, 2)          | 19.00 | <b>16.00</b> | 18.8%       | 0.78 s        |
| S8             | 5j-3s      | 5    | 3   | (2, 1, 1)       | 23.00 | <b>21.00</b> | 9.5%        | 0.70 s        |
| S9             | 4j-2s + bp | 4    | 2   | (1, 2)          | 22.00 | 22.00        | 0.0%        | 0.04 s        |
| S10            | 5j-3s + bp | 5    | 3   | (2, 2, 1)       | 24.00 | <b>19.00</b> | 26.3%       | 0.74 s        |
| <b>Average</b> |            |      |     |                 |       |              | <b>6.8%</b> | (all optimal) |

Table 6: FFLL vs. Gurobi MIP on ten small instances, all solved to proven optimality with the full C4 buffer constraints. Bold MIP values are those where FFLL fell short of the optimum. “bp” = with bypass probability  $\geq 0.15$ .

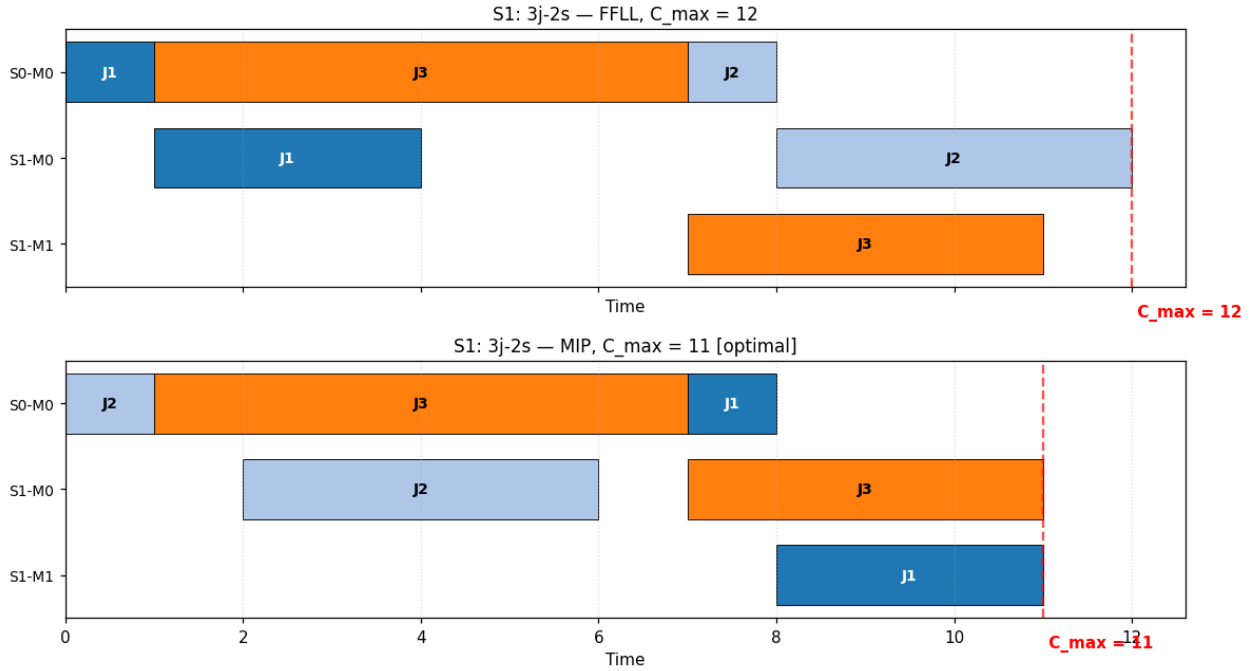


Figure 1: Gantt chart comparison on instance S1, one of the small instances the size-restricted Gurobi license could solve to optimality. Top: FFLL’s schedule, cycle time 12. Bottom: Gurobi-optimal schedule, cycle time 11 (proven optimal). Colors identify jobs, consistent across both panels. The MIP re-orders Stage-1 processing to keep Stage-2 machines continuously busy, saving one unit of cycle time.

**Comparison of solve times.** On the instances measured under the size-restricted license (S1–S3, S9), Gurobi with the full C4 buffer block solves each in a fraction of a second (0.04 s–0.27 s). FFLL, being polynomial-time by construction, runs in essentially zero time. At this scale the runtime comparison is not the interesting axis. The interesting axis is what happens beyond: the C4 block scales as  $O(n \cdot S \cdot H)$  time-indexed binary triples, so even doubling to 7–8 jobs would push MIP solve times into seconds or minutes and, further, into intractability, whereas FFLL scales gracefully. The MIP is therefore useful here as an *oracle for small instances*, giving us a rigorous quality signal for FFLL rather than a competing solver at scale.

## 4.4 Practical Implications

FFLL's per-phase decomposition and greedy sequencing leave measurable makespan on the table on a meaningful fraction of instances (five of ten here), while matching the exact optimum on the other five. The 6.8% average gap can be read either way. On the one hand, in an industrial PCB context where FFLL was originally deployed, a 7% throughput loss relative to the theoretical optimum is a meaningful margin. On the other hand, the MIP that would close that gap is intractable at anything close to the scale of a real assembly line, so in practice FFLL is unlikely to be displaced by direct optimization.

*The complete Python implementation (FFLL heuristic, Gurobi MIP model, instance generator, and comparison runner) accompanies this report as `ffll_vs_mip.py`, available at <https://github.com/annvnt/IS-16.3-Group2>.*

## References

- [1] Nicholas G. Hall and Chelliah Sriskandarajah. A survey of machine scheduling problems with blocking and no-wait in process. *Operations Research*, 44(3):510–525, 1996. doi: 10.1287/opre.44.3.510. URL <https://pubsonline.informs.org/doi/abs/10.1287/opre.44.3.510>.
- [2] S. Thomas McCormick, Michael L. Pinedo, Scott Shenker, and Barry Wolf. Sequencing in an assembly line with blocking to minimize cycle time. *Operations Research*, 37(6):925–935, 1989. doi: 10.1287/opre.37.6.925. URL <https://pubsonline.informs.org/doi/10.1287/opre.37.6.925>.
- [3] Michael L. Pinedo. *Scheduling: Theory, Algorithms, and Systems*. Springer, New York, 5th edition, 2016. doi: 10.1007/978-3-319-26580-3. URL <https://doi.org/10.1007/978-3-319-26580-3>.
- [4] Robert J. Wittrock. Scheduling algorithms for flexible flow lines. *IBM Journal of Research and Development*, 29(4):401–412, July 1985. doi: 10.1147/rd.294.0401. URL <https://ieeexplore.ieee.org/document/5390262>.